

Lógica de Programação

Prof.^a Kecia Aline Marques Ferreira

DECOM | Departamento de
Computação



Lógica de Programação

Vetores

Vetores

- Conceito
- Declaração de vetor
- Inicialização de vetores
- Acesso aos elementos do vetor
- Exemplos
- Cuidados ao usar vetor
- Mais exemplos

Conceito

Conceito

Considere o seguinte problema:

Ler o salário de 10 pessoas, calcular a média dos salários e imprimir os salários lidos.

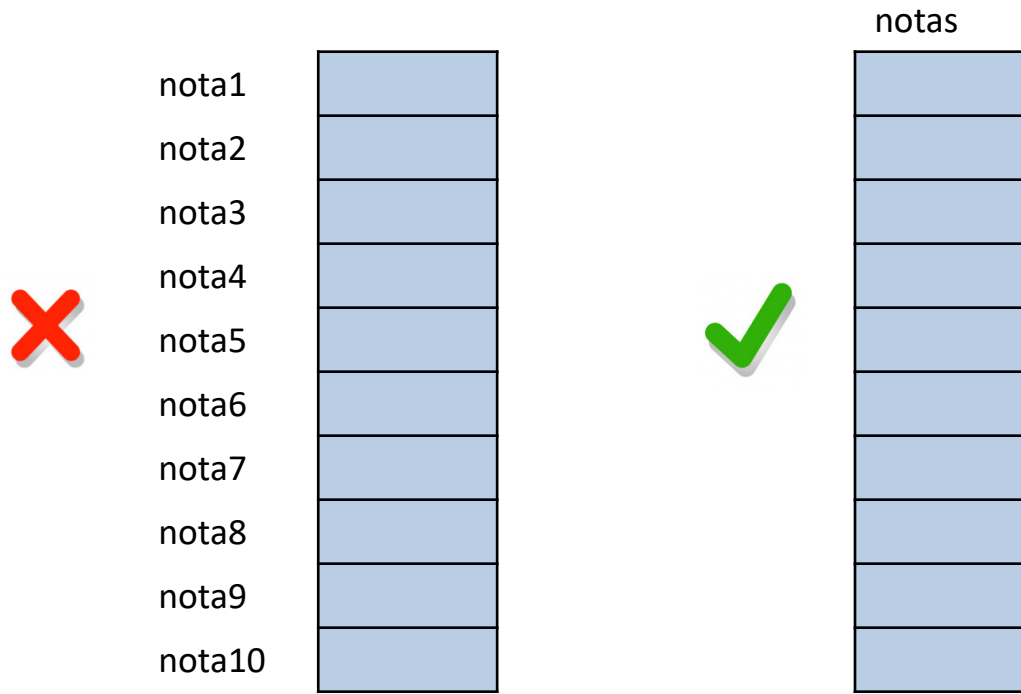
Nessa situação, será necessário armazenar os 10 salários lidos.

Declarar 10 variáveis não é um boa ideia!

Em situações como essas, usam-se os **vetores**.

Conceito

Um vetor é um tipo de dado que representa um conjunto de dados do mesmo tipo e referenciados por um só nome.



Declaração de Vetor

Declaração de Vetor

Quando o tamanho do vetor é conhecido, a sintaxe da sua declaração é a seguinte:

tipo nome[tamanho]

Onde:

tipo é um dos tipos de C ou um novo tipo definido no programa;

nome é o nome do vetor;

tamanho é a quantidade de elementos do vetor.

Exemplos:

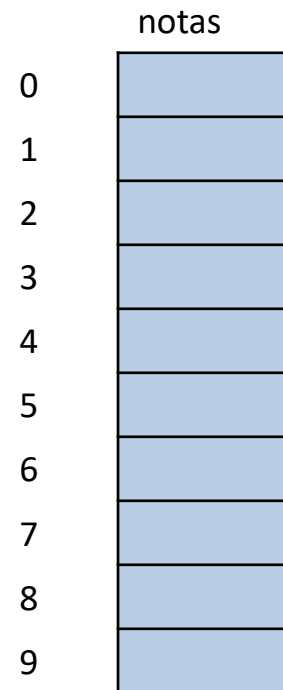
```
float notas[10];  
char nome[100];  
int idade[25];
```


Acesso aos Elementos do Vetor

Acesso aos Elementos do Vetor

Os elementos de um vetor são indexados a partir de 0 até N-1, onde N é o tamanho do vetor.

```
float notas[10];
```



Acesso aos Elementos do Vetor

Para se referenciar a um determinado elemento do vetor, utiliza-se a sintaxe a seguir:

nome[**posição**]

Onde:

nome é o nome do vetor;

posição é o índice do elemento no vetor.

Acesso aos Elementos do Vetor

```
notas[0] = 10.0;  
notas[6] = 8.0;  
  
soma = notas[0] + notas[6]
```

notas	
0	10.0
1	
2	
3	
4	
5	
6	8.0
7	
8	
9	

Acesso aos Elementos do Vetor

É útil aplicar uma variável para armazenar o índice do vetor.

Exemplo:

```
notas[i] = 10.0;
```

Nesse caso, a variável *i* é utilizada para representar a posição do vetor.

Inicialização de Vetor

Inicialização de Vetor

Se o valores de um vetor são previamente conhecidos, ele pode ser inicializado, conforme a seguinte sintaxe.

```
tipo nome[tamanho] = {valores}
```

Onde:

tipo é um dos tipos de C ou um novo tipo definido no programa;

nome é o nome do vetor;

tamanho é a quantidade de elementos do vetor;

valores é uma sequência de dados separados por vírgula.

Exemplos:

```
int idade[10] = {6 , 6, 7, 10, 8, 7, 10, 10, 9, 6};  
int diasMes[12] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
```

Inicialização de Vetor

A inicialização do vetor pode omitir o tamanho dele, desta forma:

```
tipo nome[] = {valores}
```

Onde:

tipo é um dos tipos de C ou um novo tipo definido no programa;

nome é o nome do vetor;

valores é uma sequência de dados separados por vírgula.

Exemplos:

```
int idade[] = {6 , 6, 7, 10, 8, 7, 10, 10, 9, 6};  
int diasMes[] = {31, 28, 31, 30, 31, 30, 31, 31, 30, 31, 30, 31};
```


Exemplos

Exemplos

Um programa para ler as idades de 10 pessoas e imprimir as idades lidas.

Exemplos

```
#include <stdio.h>

void main(){
    int idade[10];

    int i;

    for (i=0; i<10; i++){
        printf("\nDigite a idade: ");
        scanf("%d", &idade[i]);
    }
    printf("\n\nAs idades informadas foram: ");

    for (i=0; i<10; i++)
        printf(" %d", idade[i]);

}
```

Exemplos

Um programa para ler as idades de 10 pessoas e imprimir as idades na ordem inversa em que foram informadas.

Exemplos

```
#include <stdio.h>

void main(){
    int idade[10];

    int i;

    for (i=0; i<10; i++){
        printf("\nDigite a idade: ");
        scanf("%d", &idade[i]);
    }
    printf("\n\nAs idades informadas foram: ");

    for (i=9; i>=0; i--)
        printf(" %d", idade[i]);
}
```

Exemplos

Um programa para ler as idades de 10 pessoas e imprimir maior idade.

Exemplos

```
#include <stdio.h>

void main(){
    int idade[10], maior=0;

    int i;

    for (i=0; i<10; i++){
        printf("\nDigite a idade: ");
        scanf("%d", &idade[i]);
    }

    maior=idade[0];

    for (i=1; i<10; i++)
        if (idade[i] > maior) maior=idade[i];

    printf("\nA maior idade e %d.", maior);

}
```

Cuidados ao Usar Vetor

Cuidados ao Usar Vetor

Ao usar vetor, o programa deve sempre assegurar que não será acessada área de memória fora dos limites do vetor

Considere a seguinte situação: um vetor tem 10 posições, mas o programa tentou acessar a 11ª posição.

A linguagem C não faz essa verificação em tempo de compilação.
Essa situação é um erro de lógica que gera erro em tempo de execução!

O exemplo a seguir possui erro dessa natureza.

Exemplos

```
#include <stdio.h>

void main(){
    int idade[10];

    int i;

    for (i=0; i<10; i++){
        printf("\nDigite a idade: ");
        scanf("%d", &idade[i]);
    }
    printf("\n\nAs idades informadas foram: ");

    for (i=0; i<11; i++) //Aqui há um erro: acesso fora dos limites do vetor
        printf(" %d", idade[i]);

}
```



Mais Exemplos

Exemplos

Faça um programa para ler dois vetores de tamanho 5. O programa deverá gerar um terceiro vetor com os valores dos outros dois vetores intercalados e imprimir o vetor resultante.

Exemplos

```
#include <stdio.h>

void main(){

    int vet1[5], vet2[5], resultante[10], i, j;

    //Le o primeiro vetor
    printf("Entre com os dados do primeiro vetor: ");
    for (i=0; i<5; i++){
        scanf("%d", &vet1[i]);
    }

    //Le o segundo vetor
    printf("\n\nEntre com os dados do segundo vetor: ");
    for (i=0; i<5; i++){
        scanf("%d", &vet2[i]);
    }
```

Exemplos

```
//Intercala
for (i=0, j=0; j<10; i++, j=j+2){
    resultante[j] = vet1[i];
    resultante[j+1] = vet2[i];
}

//Imprime o vetor resultante
printf("\n\nO vetor resultante e: ");
for (i=0; i<10; i++){
    printf("%d ", resultante[i]);
}
}
```

Exemplos

Um programa para ler um vetor de 10 elementos inteiros e ordenar os dados do vetor. O programa deve imprimir o vetor resultante ordenado.

Exemplos

```
#include <stdio.h>

void main(){
    int vet[10], i, j, aux;

    //Le o vetor
    printf("Entre com os dados do vetor: ");
    for (i=0; i<10; i++){
        scanf("%d", &vet[i]);
    }

    //Ordena o vetor
    for ( i=0; i<9; i++)
        for (j=i+1; j<10; j++){
            if (vet[i]>vet[j]){
                aux = vet[i];
                vet[i] = vet[j];
                vet[j] = aux;
            }
        }
}
```


Exemplos

```
//Imprime o vetor resultante
printf("\n\nO vetor ordenado e: ");
for (i=0; i<10; i++){
    printf("%d ", vet[i]);
}

}
```